УНИВЕРСИТЕТ ИТМО

Домашняя работа №1

Проектирование базы данных

Выполнил:	Камалов Руслан Олегович
Группа:	БР1.2
ИСУ:	408732

Санкт-Петербург

2026

Отчёт по домашней работе №1

Дисциплина: Архитектура компьютерных систем (Backend) Студент: Камалов Руслан Олегович Группа: БР1.2 ИСУ: 408732

Цель

Я спроектировал реляционную базу данных фитнес-платформы FitFlow и описал сущности, связи, ключи и ограничения целостности.

Ход работы

Я выделил сущности пользователя, тренировки, плана, элемента плана, записи прогресса и статьи. Связь планов с тренировками M:N я нормализовал таблицей `plan_workouts`. Для идентификаторов я выбрал UUID, для момента выполнения - `timestamptz`, а пароль предусмотрел хранить только как bcrypt-хеш. Я добавил уникальность email и slug и индексы внешних ключей. Составной индекс истории прогресса приведён как предложение в SQL-эскизе, но автоматически не создаётся.

ER-диаграмма приведена в `docs/erd.mmd`, подробное обоснование - в `docs/homework-1-database.md`.

Диаграмма включена в приложение к отчёту и хранится в исходном виде в `docs/erd.mmd`.

Результат и вывод

Я получил схему в третьей нормальной форме, пригодную для монолита. Для последующей декомпозиции я заранее отделил домены аккаунтов, тренировок, прогресса и контента.

Приложение

ДЗ 1. Проектирование базы данных FitFlow

Предметная область

FitFlow - backend фитнес-платформы. Пользователь регистрируется, просматривает каталог тренировок, составляет личные планы, отмечает выполненные тренировки и читает статьи. Администратор управляет тренировками и статьями.

ER-диаграмма находится в `erd.mmd` (./erd.mmd) и записана в формате Mermaid.

Таблицы и ограничения

Таблица	Назначение	Основные ограничения
`users`	Учётные записи	уникальный `email`; `role` = `user` или `admin`
`workouts`	Каталог тренировок	`type` = `cardio`, `strength`, `mobility`; `level` = `beginner`, `intermediate`, `advanced`; длительность > 0
`training_plans`	Личные планы	план принадлежит одному пользователю
`plan_workouts`	Связь M:N плана и тренировок	уникальная пара (`plan_id`, `workout_id`); `position` >= 0
`progress_entries`	Факты выполнения	принадлежат пользователю и ссылаются на тренировку; длительность > 0
`articles`	Материалы платформы	уникальный `slug`; создавать может авторизованный пользователь, изменять и удалять - автор либо администратор

Все первичные ключи - UUID. Для момента выполнения используется `timestamptz`; тип автоматически создаваемых дат определяется TypeORM. Пароли не хранятся в открытом виде: сохраняется bcrypt-хеш. Числовые диапазоны и допустимые строки проверяются Zod; отдельные SQL CHECK для них пока не добавлены.

Связи

- `users 1:N training_plans` - у пользователя несколько планов;
- `training_plans M:N workouts` через `plan_workouts`;
- `users 1:N progress_entries` и `workouts 1:N progress_entries`;
- `users 1:N articles`.

В монолите внешние ключи должны обеспечиваться PostgreSQL. После разделения на микросервисы межсервисные UUID остаются логическими ссылками без распределённых внешних ключей.

Индексы

- уникальные: `users(email)`, `articles(slug)`, `plan_workouts(plan_id, workout_id)`;
- поисковые: UUID-колонки внешних ключей; составные `workouts(type, level)` и `progress_entries(user_id, completed_at)` предложены ниже, но в TypeORM-сущностях пока отсутствуют;
- сортировка каталога и статей выполняется по `created_at`.

Нормализация и целостность

Схема соответствует третьей нормальной форме: повторяющиеся группы отсутствуют, M:N вынесена в отдельную таблицу, неключевые поля зависят от ключа целиком. Удаление пользователя не должно удалять каталог тренировок; личные планы и прогресс удаляются каскадно. Удаление тренировки запрещается при наличии ссылок либо выполняется как контролируемое удаление после очистки связей.

SQL-эскиз дополнительных ограничений (не применяется автоматически)

```
ALTER TABLE workouts ADD CONSTRAINT workouts_duration_positive CHECK (duration_minutes >
 0);
ALTER TABLE progress_entries ADD CONSTRAINT progress_duration_positive CHECK
(duration_minutes > 0);
ALTER TABLE plan_workouts ADD CONSTRAINT plan_position_nonnegative CHECK (position >=
0);
CREATE INDEX progress_user_completed_idx ON progress_entries(user_id, completed_at
DESC);
CREATE INDEX workout_filters_idx ON workouts(type, level);
```

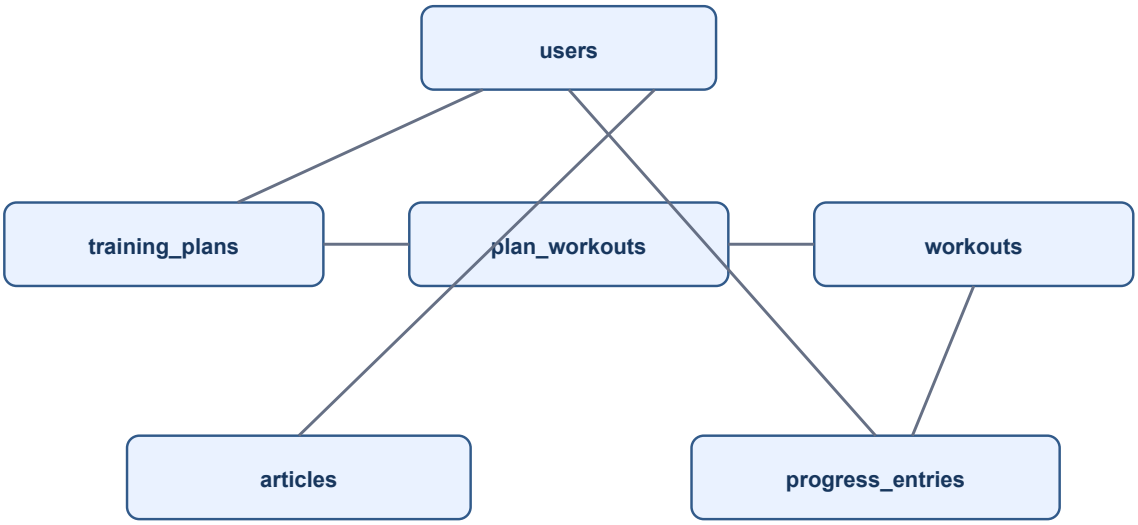


Рисунок 1 — логическая ER-модель FitFlow